Installation

Nix and NixOS — the supported installation

```
nix run . -- path/to/deck.pdf        # or: nix profile install .
nix develop                          # dev shell, then: cargo run -- deck.pdf
make launch                          # development launcher
make launch DECK=examples/mosaic.pdf # development launcher, Mosaic example
```

The packaged binary is wrapped with the loader path it needs and a pinned PDFium, so it starts with no environment setup at all. This matters on NixOS specifically: winit, wgpu and PDFium are all `dlopen`ed at run time, and a plain `cargo install` binary finds nothing without a global `/usr/lib`.

Other distributions

```
./scripts/fetch-pdfium.sh            # pinned, hash-verified PDFium into ./lib
cargo run --release -- path/to/deck.pdf
```

This needs the usual desktop libraries present (`libxcursor`, `libxkbcommon`, a Vulkan loader). If they are missing, Pulpit says which ones and how to install them rather than crashing.

`.deb` and `.rpm` packages are the planned Linux install and do not exist yet. They are the right answer because they can declare what Pulpit actually needs — the desktop libraries as dependencies, and a Chromium-family browser as a *recommendation*, so media overlays work on a default install instead of falling back to posters. Flatpak, Snap and AppImage are deliberately not targets: a sandbox blocks the browser Pulpit has to launch as a child process, and a self-contained image cannot carry the graphics drivers, which are the part that actually breaks. Until the native packages exist, `make install` and `scripts/make-bundle.sh` are best-effort helpers and not a tested surface.

Windows

Through a package manager, which is the intended channel:

```
winget install VincentArelBundock.Pulpit
scoop install pulpit
```

Or download the installer from the releases page. It is **per-user**: it writes under `%LOCALAPPDATA%` and raises no UAC prompt, so no administrator account is involved. A portable `.zip` is published alongside it for anyone who would rather not install anything.

Windows needs less from a package than the other two desktops. The graphics stack ships with the operating system, `pdfium.dll` sits beside `pulpit.exe` where the search order already looks, and Edge is preinstalled and Chromium-family, so media overlays work without installing a browser.

Builds are not signed. Signing is a quality improvement here rather than a prerequisite: winget verifies a checksum and Scoop needs neither, so an install through a package manager does not take the browser-download path that produces the loudest SmartScreen warning. Downloading the installer in a browser may still show one; *More info* then *Run anyway* dismisses it. If free signing for open source is obtained, the signature appears and nothing else about installing changes.

x64 only at present. Windows on ARM runs the x64 build under emulation.

macOS

```
brew install --cask vincentarelbundock/tap/pulpit
```

Or download the disk image from the releases page and drag `Pulpit.app` onto Applications. Either way it carries its own `libpdfium`, and the graphics stack is the operating system's, so there is nothing else to install.

The app is **ad-hoc signed but not notarized**. Ad-hoc signing is what lets Apple Silicon execute the binary at all; it buys no Gatekeeper relief. Notarization needs a paid Developer ID, and no release is gated on one, so the first launch after a browser download costs one trip through System Settings → Privacy & Security, where a message about Pulpit offers *Open Anyway*. macOS 15 removed the Control-click shortcut that used to make this quicker. It happens once, not once per version.

The prompt comes from the `com.apple.quarantine` attribute, which the *downloading* program sets — browsers do, command-line tools do not. Installing through Homebrew therefore skips it, provided the quarantine is not re-applied:

```
brew install --cask --no-quarantine vincentarelbundock/tap/pulpit
```

Homebrew re-applies quarantine to cask installs deliberately, so `--no-quarantine` is the documented way to opt out rather than something to assume.

Builds are Apple Silicon only at present. Displays are identified through CoreGraphics, which reports a vendor/model/serial triple taken from the panel itself, so a projector keeps its identity across a re-plug and a different port without an EDID parse.

PDFium

`libpdfium` is required, not optional. Every package installs it; if it is missing, Pulpit tells you where it looked and exits, rather than showing you placeholder pages where your slides should be.

PDFium is not vendored in this repository and not linked into the binaries: it is loaded dynamically at run time, so a build without it still succeeds and the application degrades visibly rather than silently.

Binary acquisition

`scripts/fetch-pdfium.sh` downloads a pinned release from `bblanchon/pdfium-binaries` and verifies its SHA-256 before installing it into `./lib`. Treat that project as an **unaffiliated third-party supply-chain dependency**:

- The release tag (`chromium/NNNN`) and the per-target SHA-256 are pinned in the script. Never replace them with "latest".
- A target with no recorded hash refuses to install and prints the observed hash for review. Verify independently before pinning it.
- Archive the downloaded artefact alongside release inputs so a build can be reproduced after the upstream release is deleted.
- Review upstream changes (PDFium version, build flags, third-party notices) when bumping.

Currently pinned: `chromium/7999`, `pdfium-linux-x64.tgz`, `c3af580f9df0fef9545b44115bc5ea440f286956b5f231df69fb373b8efc4f69`.

If the service disappears, PDFium can be built from source with `depot_tools` and the same GN args the upstream project publishes (`args.gn` ships inside each artefact and is worth archiving). The application only needs a shared library exporting the standard `FPDF_*` symbols; nothing in this codebase depends on that project specifically.

PDFium is BSD-3-Clause. Redistribution obligations, including the bundled third-party notices shipped as `lib/PDFIUM-LICENSE`, are release requirements and must be included in any package that ships the library.

Run-time discovery

Search order in `PdfiumBackend::bind`:

1. `PULPIT_PDFIUM_PATH` (a file or a directory)
2. the directory containing the executable, and `<exe dir>/lib`
3. `./lib` and `.`
4. the system loader path

Failure at every step is reported once, listing the paths tried, and the renderer worker exits: PDFium ships with every supported package, so this is a broken installation and placeholder pages on the projector would be worse than stopping. `PULPIT_FORCE_FIXTURE_BACKEND=1` selects the fixture backend explicitly, which is how the tests run without PDFium.

Testing an installation

```
cargo test                                  # everything; no display required
cargo test -p pulpit-display --test topology_script
sudo ./scripts/vkms-topology.sh             # virtual connectors, privileged
```

- Reconciliation, presentation state, document reload, cache accounting, render generations and protocol decoding are pure tests.
- **Scripted topologies**: every file in `crates/pulpit-display/tests/topology/` is replayed through the real state machine under X11, Wayland and tiling capability profiles, asserting the invariants after each transition. `pulpit-topology` dumps a live topology in that same format, so a session with an awkward dock or projector becomes a permanent regression test by committing the capture.
- Supervisor tests spawn **real worker processes**, including a deliberately crashing and a deliberately hanging one.
- PDFium tests render a generated PDF and skip with a message when no `libpdfium` is installed.